Practical # 12

Objective:
Develop Merge Sort and Quick Sort; evaluate performance on large datasets.

Theory:
Sorting techniques (i.e., Merge Sort, Quick Sort, and Radix Sort refer to the operation or technique of arranging and rearranging sets of data in some specific order.

Merge Sort works:
Merge Sort Algorithm works in the following steps:
Step 1 - It divides the given unsorted array into two halves- left and right sub arrays.
Step 2 - The sub arrays are divided recursively.
Step 3 - This division continues until the size of each sub array becomes 1.
Step 4 - After each sub array contains only a single element, each sub array is sorted trivially.
Step 5 - Then, the merge procedure combines these trivially sorted arrays to produce a final sorted array.

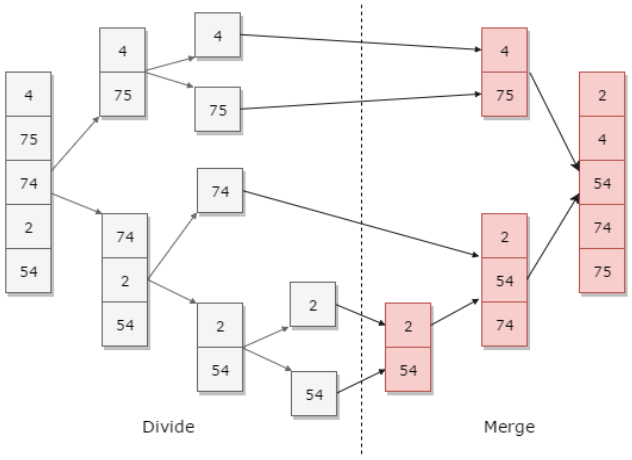

Fig: Merge Sort Technique

In this lab, students will learn how to:

- Develop Merge Sort and Quick Sort algorithms in C++.
- Implement the divide-and-conquer approach for sorting.
- Analyze the time and space complexity of Merge Sort and Quick Sort.
- Evaluate performance differences between Merge Sort and Quick Sort on large datasets.
- Compare recursive and iterative approaches where applicable.

C++ program: Write C++ program to arrange elements in ascending order using Merge sorting algorithm.

```
#include <iostream>
using namespace std;

void merge(int *,int, int , int );
void mergesort(int *a, int low, int high)
{
     int mid;
     if (low < high)
     {
mid=(low+high)/2;
mergesort(a,low,mid);
mergesort(a,mid+1,high);
merge(a,low,high,mid);
     }
     return;
}
// Merge sort concepts starts here
void merge(int *a, int low, int high, int mid)
{
     int i, j, k, c[50];
     i = low;
     k = low;
     j = mid + 1;
     while (i <= mid && j <= high)
     {
if (a[i] < a[j])
{
     c[k] = a[i];
     k++;
     i++;
}
else
{
     c[k] = a[j];
     k++;
     j++;
}
     }
     while (i <= mid)
     {
c[k] = a[i];
k++;
i++;
     }
     while (j <= high)
     {
c[k] = a[j];
k++;
j++;
     }
     for (i = low; i < k; i++)
     {
a[i] = c[i];
     }
}
// from main mergesort function gets called
int main()
{
     int a[30], i, b[30];
     cout<<"enter  five elements (unsorted array):\n";
     for (i = 0; i < 5; i++) { cin>>a[i];
     }
     mergesort(a, 0, 4);
     cout<<"sorted array\n";
     for (i = 0; i < 5; i++)
     {
cout<<a[i]<<"\t";
     }
}
```

```
enter  five elements (unsorted array):
45
12
32
2
5
sorted array
2       5       12      32      45
```

Review Questions/ Exercise:

1. Write a C++ program to implement Merge Sort for an integer array.
2. Write a C++ program to implement Quick Sort using the first element as pivot.
3. Modify Quick Sort to use the last element as pivot and test performance.
4. Implement Quick Sort with a random pivot selection strategy.
5. Implement Merge Sort to sort an array of strings alphabetically.

Name: _____

Roll #: _____

Date: _____

Subject Teacher